

Transfer Learning and Error Analysis for Binary Food Image Classification

Xutong Dong

Zhaoning Liang

Yirong Li

Jiaxu Zhang

May 2026

Abstract

This paper summarizes a reproducible binary food image classification study that distinguishes HEALTHY and UNHEALTHY food images. The project compares a small convolutional neural network trained from scratch, an ImageNet-pretrained ResNet18 with a frozen backbone, fully fine-tuned ResNet18 variants, and a supervised contrastive pretraining branch. On a balanced held-out test set of 522 images, the best observed model is a ResNet18 initialized from a frozen-transfer checkpoint and fully fine-tuned with learning rate 10^{-4} , achieving 0.9617 accuracy, 0.9617 macro F1, and 0.9944 ROC AUC. A focused ablation indicates that adapting high-level ResNet features accounts for most of the improvement over a frozen backbone, while full-backbone fine-tuning at the largest tested learning rate performs best. Error analysis shows 20 residual test errors, split evenly across the two classes.

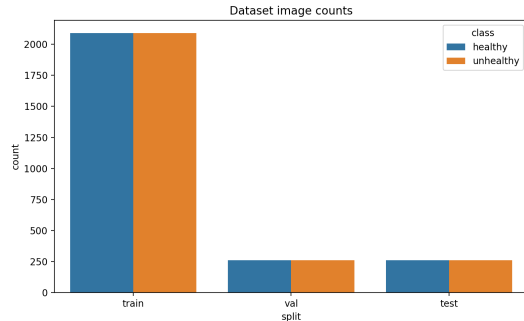
1 Introduction

Image-based food classification is a practical visual recognition problem in which object appearance, plating context, and class definitions can all introduce ambiguity. This project studies a binary version of the task: each image is assigned to either HEALTHY or UNHEALTHY. The goal is not to claim nutritional ground truth beyond the dataset labels, but to evaluate whether modern transfer learning methods can produce reliable predictions under a fixed train/validation/test protocol.

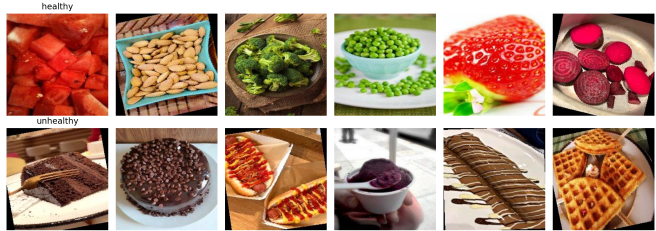
The project is organized as an incremental empirical study. It begins with exploratory data analysis and a compact convolutional neural network (CNN) baseline, then compares the baseline with ImageNet-pretrained ResNet18 transfer learning. The transfer model is evaluated both with a frozen feature extractor and with subsequent fine-tuning. A supervised contrastive learning branch is also included to test whether label-aware representation learning improves the small CNN encoder. Finally, the strongest ResNet18 configuration is examined through a focused ablation and an error analysis of held-out test predictions. The repository `README.md` documents the pipeline commands, generated artifacts, and reproduction workflow used to produce the summaries cited in this paper [7]. Team collaboration and code versioning were managed through GitHub at <https://github.com/JiaxuZhang-03/I242BProject> [2].

The current dataset contains 5,222 images with balanced class counts in all splits: 4,178 training images, 522 validation images, and 522 test images. Because the two classes are balanced, accuracy, macro F1, and weighted F1 are directly comparable for the main evaluation. The best model observed in the generated results is a fully fine-tuned ResNet18 initialized from a frozen-transfer checkpoint. It reaches 0.9617 test accuracy and 0.9944 ROC AUC, improving substantially over both the simple CNN baseline and the frozen-backbone transfer model.

Figure 1 shows the split-level class counts and a sample grid generated during exploratory analysis.



(a) Class counts by split.



(b) Training-set examples.

Figure 1: Exploratory data analysis artifacts from `result/eda`. The dataset is balanced across HEALTHY and UNHEALTHY labels in the training, validation, and test splits.

2 Related Work

Convolutional neural networks remain a standard architecture family for image recognition because they combine local receptive fields with hierarchical feature learning. Residual networks improve optimization by introducing skip connections, which allow deeper CNNs to learn residual mappings rather than entirely new transformations at each layer [3]. In this project, ResNet18 is used as a computationally modest transfer-learning backbone.

Transfer learning is especially useful when task-specific datasets are smaller than the datasets used to train general-purpose visual encoders. ImageNet pretraining provides features learned from a large and diverse image collection [1]. The project evaluates two common transfer-learning regimes: training only a new classification head while freezing the backbone, and unfreezing part or all of the pretrained backbone for task-specific fine-tuning.

The supervised contrastive learning branch follows the idea that examples sharing a label should be close in representation space, while examples from different labels should be separated [4]. This objective is tested as an alternative representation-learning strategy for the simple CNN encoder. The resulting encoder is then fine-tuned for binary classification.

For interpretability, the project generates Grad-CAM visualizations [8]. These maps are not treated as causal explanations, but they provide a qualitative check on which image regions influence the selected ResNet18 model. The implementation uses PyTorch for training and evaluation [6], AdamW for optimization [5], and standard image augmentations consistent with modern CNN training practice [9].

3 Method

3.1 Problem Setup

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ denote the image dataset, where x_i is an RGB food image and $y_i \in \{0, 1\}$ indicates the class label. The model produces logits $z_i = f_\theta(x_i) \in \mathbb{R}^2$, and class probabilities are computed by

$$p_\theta(y = c \mid x_i) = \frac{\exp(z_{ic})}{\sum_{k=1}^2 \exp(z_{ik})}. \quad (1)$$

Table 1: Dataset split sizes from `result/eda/dataset_counts.csv`.

Split	Healthy	Unhealthy	Total
Train	2089	2089	4178
Validation	261	261	522
Test	261	261	522

Classifier training minimizes the regularized empirical risk

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n L_{\text{ce}}(f_{\theta}(x_i), y_i) + \lambda \|\theta_{\text{train}}\|_2^2, \quad (2)$$

where L_{ce} is cross-entropy and θ_{train} denotes the parameters updated in a given experiment. In implementation, the L_2 -style penalty is applied through AdamW weight decay. Dropout in the classifier head, data augmentation, cosine learning-rate decay, and validation-based early stopping provide additional regularization.

3.2 Data Processing

The project uses an ImageFolder-style directory layout with separate `train`, `val`, and `test` folders. Images are resized to 128×128 pixels for the current reported experiments. Training uses random resized crops, horizontal flips, and color jitter. Validation and test evaluation use deterministic resizing followed by center cropping. All images are normalized using ImageNet channel statistics, which is appropriate for the pretrained ResNet18 branch.

3.3 Models

The simple CNN baseline uses four convolutional blocks with batch normalization, ReLU activations, and max pooling in the first three blocks. Adaptive average pooling converts the final feature map into a 256-dimensional feature vector. A dropout layer and linear classifier map this feature vector to two logits.

The transfer-learning models use a ResNet18 backbone. In the frozen-backbone configuration, ImageNet weights initialize the encoder and only the final binary classification head is trained. In the fine-tuning configurations, the model is initialized from the frozen-transfer checkpoint and then trained with either only ResNet `layer4` unfrozen or the full backbone unfrozen. The ablation varies the trainable scope and learning rate.

3.4 Advanced Extension: Supervised Contrastive Branch

The advanced methodological extension in this project is supervised contrastive pretraining, which is evaluated as a representation-learning alternative to direct cross-entropy training. For this branch, each training image is transformed into two augmented views. Let u_i and v_i denote normalized projected embeddings. The supervised contrastive loss encourages embeddings with the same label to have high cosine similarity and embeddings with different labels to have lower similarity:

$$\mathcal{L}_{\text{supcon}} = \sum_i \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(u_i, v_p)/\tau)}{\sum_{a \neq i} \exp(\text{sim}(u_i, v_a)/\tau)}, \quad (3)$$

Table 2: Held-out test results from `result/experiment_summary.csv`.

Experiment	Best val. acc.	Test acc.	Macro F1	ROC AUC
ResNet18 full fine-tune, LR 10^{-4}	0.9655	0.9617	0.9617	0.9944
ResNet18 layer4 fine-tune, LR 3×10^{-5}	0.9540	0.9406	0.9406	0.9850
ResNet18 full fine-tune, LR 3×10^{-5}	0.9598	0.9406	0.9406	0.9845
ResNet18 full fine-tune, LR 10^{-5}	0.9406	0.9310	0.9310	0.9781
ResNet18 frozen backbone	0.8563	0.8697	0.8697	0.9344
Simple CNN + SupCon fine-tune	0.8084	0.8046	0.8033	0.8813
Simple CNN baseline	0.7989	0.8027	0.8027	0.8869

where $P(i)$ is the set of same-label positives for anchor i , $\text{sim}(\cdot, \cdot)$ is cosine similarity after normalization, and $\tau = 0.07$ in the current implementation. The pretrained encoder is then reused in a classifier and fine-tuned with cross-entropy.

3.5 Evaluation

Each classifier is evaluated on the held-out test set. The generated summaries report accuracy, macro F1, weighted F1, and ROC AUC. For binary classification, the confusion matrix is

$$C_{ab} = |\{i : y_i = a, \hat{y}_i = b\}|, \quad (4)$$

where a is the true class and b is the predicted class. Error analysis additionally records misclassified samples, predicted-class confidence, class-specific error rates, and high-confidence errors.

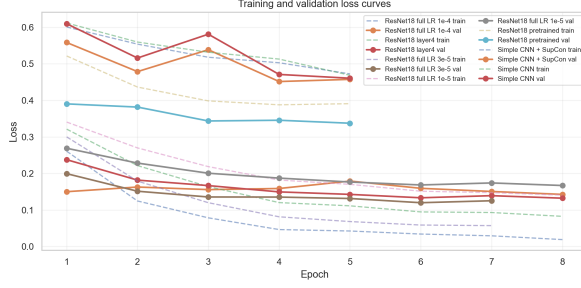
4 Experiments

4.1 Training Protocol

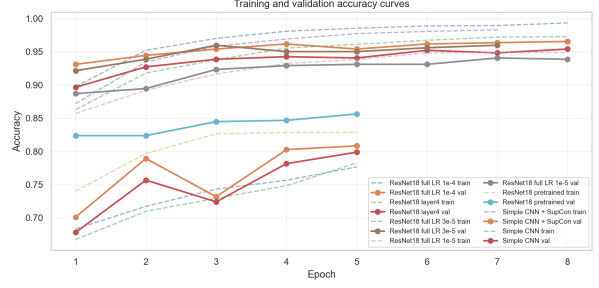
All reported model runs use 128×128 inputs and the balanced train/validation/test split in Table 1. Classifiers are optimized with AdamW and cosine learning-rate scheduling. The baseline simple CNN, frozen ResNet18, and SupCon fine-tuned classifier are trained for five epochs in the current summary. ResNet18 fine-tuning runs for up to eight epochs, with early stopping available in the training script.

Table 2 shows that ImageNet transfer improves substantially over the simple CNN baseline, and that full ResNet18 fine-tuning gives the strongest result. The best model improves test accuracy from 0.8697 for the frozen-backbone ResNet18 to 0.9617, an absolute gain of 0.0920. It also improves ROC AUC from 0.9344 to 0.9944.

Figure 2 provides the main bias-variance diagnostic. The simple CNN and SupCon-initialized simple CNN remain below the ResNet18 variants, suggesting that the compact encoder underfits the task relative to the transferred residual backbone. The best full fine-tuned ResNet18 reaches high training accuracy while validation accuracy continues to improve through the final epoch, so the saved curves do not show severe validation collapse. The gap between training and validation loss nevertheless grows during fine-tuning, which indicates that regularization and validation monitoring remain important for this dataset.

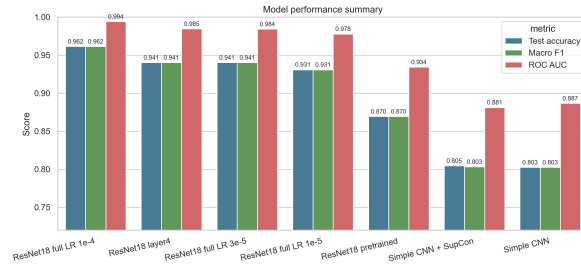


(a) Training and validation loss.

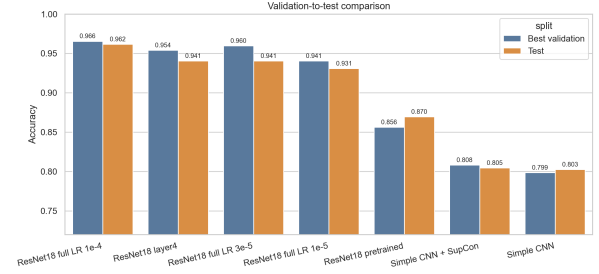


(b) Training and validation accuracy.

Figure 2: Optimization diagnostics generated from the saved history files.



(a) Model metric comparison.



(b) Validation and test accuracy.

Figure 3: Summary plots generated from the experiment outputs in `result/summary_plots`.

4.2 Ablation

The ResNet18 ablation separates the effects of initialization, trainable scope, and learning rate. Training only the classifier head provides a useful transfer baseline but reaches only 0.8697 test accuracy. Unfreezing ResNet **layer4** raises accuracy to 0.9406, indicating that high-level feature adaptation explains much of the gain. Full-backbone fine-tuning is sensitive to the learning rate: 10^{-5} under-adapts in the current run, 3×10^{-5} matches the layer4 setting, and 10^{-4} achieves the best test accuracy.

4.3 Error Analysis

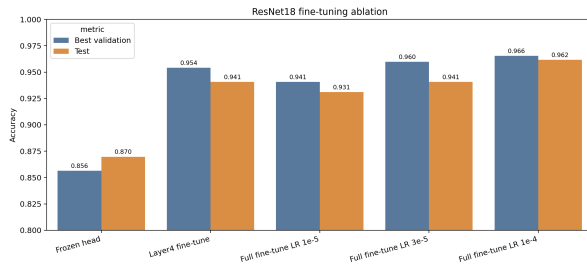
The best model makes 20 errors on 522 test images. Errors are balanced across classes: 10 HEALTHY images are predicted as UNHEALTHY, and 10 UNHEALTHY images are predicted as HEALTHY. The resulting confusion matrix is shown in Table 4. The mean predicted-class confidence among errors is 0.7843, and 12 of the 20 errors have confidence at least 0.80. These high-confidence failures are useful targets for qualitative inspection because they are not simply low-confidence borderline predictions.

4.4 Missing or Future Results

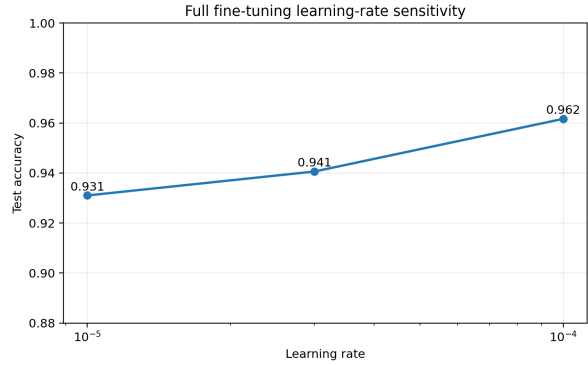
The repository contains the main summaries, ablation outputs, error-analysis tables, and visual artifacts needed for the current report. Additional experiments, such as longer SupCon pretraining, larger backbones, confidence calibration, or cross-dataset validation, have not been run in the provided artifacts. For that reason, this report does not claim results for those extensions.

Table 3: Focused ResNet18 fine-tuning ablation from `result/ablation_summary.csv`.

Setting	Trainable scope	LR	Test acc.	ROC AUC	Trainable params
Frozen head	classifier head	0.0010	0.8697	0.9344	1026
Layer4 fine-tune	classifier + layer4	0.0000	0.9406	0.9850	8394754
Full fine-tune	full backbone	0.0000	0.9310	0.9781	11177538
Full fine-tune	full backbone	0.0000	0.9406	0.9845	11177538
Full fine-tune	full backbone	0.0001	0.9617	0.9944	11177538



(a) Ablation accuracy.



(b) Learning-rate sensitivity.

Figure 4: ResNet18 ablation plots generated from `result/ablation_summary.csv`.

5 Safety, Security, and Ethics

The model should not be deployed as an authoritative nutrition or health advisor. The labels HEALTHY and UNHEALTHY simplify a complex nutritional judgment that depends on ingredients, portion size, preparation, culture, dietary constraints, and individual medical context. A visually plausible prediction can therefore be misleading even when the classifier is accurate on the held-out dataset.

At large scale, the main safety risk is over-reliance: users or organizations might treat the output as dietary guidance rather than a coarse visual categorization. The observed 0.9617 test accuracy still corresponds to 20 errors on 522 held-out images, including 12 high-confidence errors. A production system would need calibrated uncertainty, human review for high-impact uses, clearer label definitions, and evaluation on data collected from the intended deployment population.

There are also fairness and privacy concerns. Food appearance varies by cuisine, region, socioeconomic context, and image quality; a model trained on one dataset may perform worse on underrepresented foods or presentation styles. If user-uploaded meal images were collected, the system would need safeguards for consent, retention, access control, and removal of incidental personal information. For these reasons, the current project is best viewed as a controlled academic study and not as a ready-to-deploy health tool.

6 Conclusion

This project demonstrates that transfer learning is the dominant performance driver for the binary food image classification task studied here. A simple CNN trained from scratch reaches 0.8027

Table 4: Confusion matrix for the best ResNet18 model on the test set.

True class	Predicted healthy	Predicted unhealthy
Healthy	251	10
Unhealthy	10	251

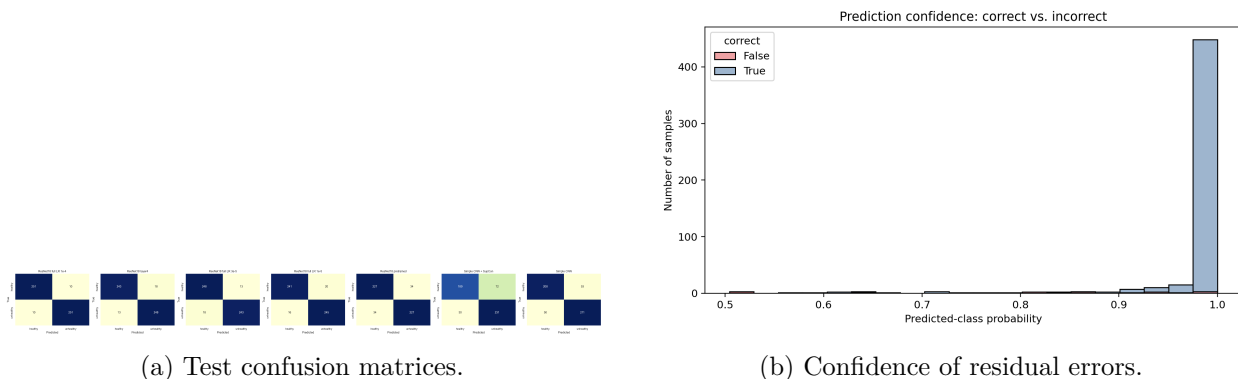


Figure 5: Evaluation and error-analysis plots from `result/summary_plots`.

test accuracy, while a frozen ImageNet-pretrained ResNet18 improves to 0.8697. Fine-tuning the pretrained ResNet18 provides the largest gain, with the best current configuration reaching 0.9617 test accuracy and 0.9944 ROC AUC.

The ablation suggests that adapting high-level residual features is already sufficient to capture much of the improvement, but full-backbone fine-tuning with learning rate 10^{-4} performs best among the tested settings. The final error analysis shows balanced residual errors across HEALTHY and UNHEALTHY classes, with several high-confidence mistakes that warrant qualitative review. Future work should avoid reporting speculative gains and instead extend the present pipeline with new generated summaries for longer training, alternative backbones, calibration analysis, or evaluation on an independently collected dataset.

References

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [2] Dong, Xutong and Liang, Zhaoning and Li, Yirong and Zhang, Jiaxu. I242bproject github repository. <https://github.com/JiaxuZhang-03/I242BProject>, 2026. Code repository used for team collaboration and version control.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [4] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673, 2020.

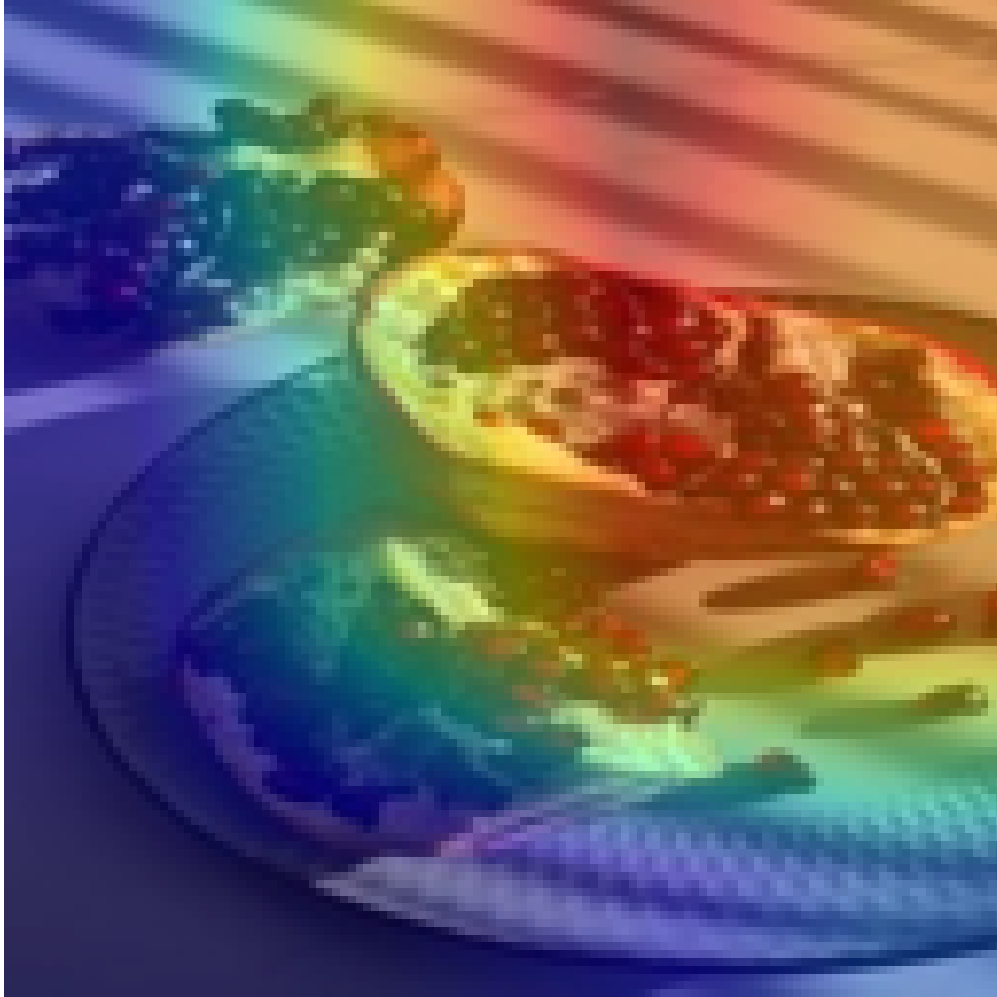


Figure 6: Example Grad-CAM artifact for the selected ResNet18 model. Additional Grad-CAM images are stored under `result/gradcam`. This figure is qualitative and is not used as quantitative evidence.

- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- [6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [7] Project Repository. Indeng 1/242b spring 2026 project readme. <https://github.com/JiaxuZhang-03/I242BProject>, 2026. Project documentation describing the dataset, pipeline, commands, and generated artifacts.
- [8] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-

based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 618–626, 2017.

- [9] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

A Code and Reproducibility

The project code is organized as executable scripts and reusable modules. The repository `README.md` documents the full workflow. The team used GitHub for code collaboration, version control, and project sharing: <https://github.com/JiaxuZhang-03/I242BProject>.

- `src/train_classifier.py`: classifier training, transfer learning, checkpoint initialization, and fine-tuning policies.
- `src/pretrain_supcon.py`: supervised contrastive pretraining.
- `src/evaluate_model.py`: held-out evaluation, metrics, predictions, ROC curves, and confusion matrices.
- `src/error_analysis.py`: residual error summaries and high-confidence error analysis.
- `src/food_project/models.py`: simple CNN, ResNet18 classifier wrappers, and SupCon projection head.
- `src/food_project/data.py`: dataset discovery, transformations, dataloaders, and split summaries.

From the repository root, the complete pipeline can be reproduced with:

```
./download_data.sh
./setup_environment.sh
./run_full_pipeline.sh
./run_resnet_ablation.sh
```

Representative source excerpts are included below; the complete implementation is in the repository.

A.1 Model Definition Excerpt

```
    resnet18,
    resnet50,
)

MODEL_NAMES = (
    "simple_cnn",
    "resnet18",
    "resnet50",
    "mobilenet_v3_small",
    "efficientnet_b0",
)

class SimpleCNNEncoder(nn.Module):
```

```

def __init__(self):
    super().__init__()
    self.feature_dim = 256
    self.features = nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3, padding=1),
        nn.BatchNorm2d(32),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),
        nn.Conv2d(128, self.feature_dim, kernel_size=3, padding=1),
        nn.BatchNorm2d(self.feature_dim),
        nn.ReLU(inplace=True),
    )
    self.pool = nn.AdaptiveAvgPool2d((1, 1))

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.features(x)
    x = self.pool(x)
    return torch.flatten(x, 1)

class ClassifierModel(nn.Module):
    def __init__(
        self,
        encoder: nn.Module,
        feature_dim: int,
        num_classes: int,
        dropout: float = 0.2,
    ):
        super().__init__()
        self.encoder = encoder
        self.feature_dim = feature_dim

```

A.2 Fine-Tuning Policy Excerpt

```

def configure_trainable_backbone(model: nn.Module, policy: str) -> None:
    if policy == "all":
        return

    for parameter in model.encoder.parameters():
        parameter.requires_grad = False

    if policy == "none":
        return

    layer_names = {
        "resnet-layer4": ("layer4",),
        "resnet-layer3-layer4": ("layer3", "layer4"),
    }[policy]
    missing_layers = [name for name in layer_names if not hasattr(model.encoder, name)]
    if missing_layers:
        raise ValueError(
            f"Policy {policy!r} requires ResNet-style layers, missing {missing_layers}."
        )

    for name in layer_names:
        for parameter in getattr(model.encoder, name).parameters():
            parameter.requires_grad = True

```

```
def main() -> None:
    args = parse_args()
    if args.init_checkpoint is not None and args.supcon_checkpoint is not None:
        raise ValueError("Use either --init-checkpoint or --supcon-checkpoint, not both.")

    set_seed(args.seed)
    device = get_device(args.device)
```